A Computation Model for Z
Based On Concurrent Constraint Resolution

Wolfgang Grieskamp

Technische Universität Berlin, FB13, Institut für Kommunikations- und Softwaretechnik.
Sekr. 5–13, Franklinstr. 28/29, D–10587 Berlin.
E-mail: wg@cs.tu-berlin.de

Abstract. We present a computation model for Z, which is based on a reduction to a small calculus, called μZ, and on concurrent constraint resolution techniques applied for computing in this calculus. The power of the model is comparable to that of functional logic languages, and combines the strength of higher-order functional computation with logic computation. The model is implemented as part of the ZETA system, where it is used for executing Z specifications for the purpose of test-data evaluation and prototyping.

1 Introduction

The automatic evaluation of *test data* for *safety-critical systems* is an interesting application that can help to put formal methods into industrial practice. Some studies report that more than 50% of development costs in this application area go into testing. A setting for test-case evaluation that can improve this situation is as follows: given a requirements specification, some input data describing a test case, and the output data from a run of the system's implementation on the given input, we check by executing the specification whether the implementation meets its requirements. At the first sight, this goal would seem to be simple, since input and output data are fixed. However, a real-world specification of a complex system may contain a lot of "hidden" data, which is used to describe the observable behavior. Thus, the problem scales up to finding solutions to (a sequence of) partial data bindings. Moreover, for embedded systems, a run over a few minutes may easily produce sequences of several thousand data bindings, making the execution of the specification time-critical, such that symbolic proving techniques are hard to apply.

In the application-oriented research project ESPRESS[1], which is concerned with methods and tools for the development of embedded systems, the set-based specification language Z [20] is used for requirements specification in combination with other notations such as Statecharts which are incorporated by a shallow encoding in Z [5]. Performing test-case evaluation in this setting requires a *computation model* for Z.

In this paper, we report on some results of the efforts on defining and implementing such a model. The main theme of the paper is the presentation of the μZ calculus and the

[1] ESPRESS was a joint project involving industrial partners (Daimler-Benz AG and Robert Bosch AG) and academic research institutes (GMD-First, FhG-ISST, and TU-Berlin), funded by the German "Bundesministerium für Bildung, Wissenschaft, Forschung und Technologie".

definition of a computation model for it. The μZ calculus is a pure expression language with only a few constructs which abstracts from the diversity of Z. Since it can embed the full Z language – the simple-typed λ-calculus, set calculus, predicate calculus, and schema calculus – computation in μZ can indeed not be complete: but a computation model and implementation by an abstract machine can be provided which is comparable to that of functional logic paradigms (see [10] for on overview on the functional logic paradigm).

The work presented in this paper describes a refinement of an existing compiler for Z, which is part of the ESPRESS tool environment ZETA [4]. Within the old compiler, Z is reduced to an earlier version of μZ, and μZ is compiled to Java, using a functional (deterministic) approach to execution. Case studies performed together with industry partners demonstrated the feasibility of the principal approach, but also clearly showed that deterministic computation is a serious restriction for test-data evaluation, since it constraints the way of how requirement specifications can be formulated in Z significantly. Thus this work defines an enhanced computation model for μZ, incorporating indeterministic logic principles. For the enhanced model we report on, an abstract machine has been developed [8, 9], and a high-performance implementation in C++ has been derived, which is part of the ZETA release since version 1.5.

This paper is organized as follows. In Sec. 2 an introduction to the μZ calculus is given. Sec. 3 defines the mapping of central Z concepts to μZ. Sec. 4 sketches the model semantics of μZ. Sec. 5 provides a complete description of the computation model of μZ in the style of natural semantics. In Sec. 6, finally, we give some examples of executable Z constructs, analyze the power of the computation model, and point out possible enhancements, whereas in Sec. 7 we discuss related work and provide the conclusion.

2 The μZ Calculus: Syntax and Informal Semantics

The μZ calculus is a set-based expression language which can embed notations such as Z as well as functional and logical programming languages. The calculus is designed for the automatic analysis and transformation, where it is desirable working with a *small* set of language constructs. However, μZ is not "as small as possible"[2]. The calculus has redundancy in that it allows *extensional* as well as *intensional* description of the same meaning. For example, the set containing the numbers 1 and 2 can be denoted in μZ as $\{1\} \cup \{2\}$ or by the set comprehension $\{x \mid (x, 1) \in (_ \geq _) \wedge (x, 2) \in (_ \leq _)\}$. The support of both intensional and extensional description allows the definition of computation as a source-level transformation from intensional towards extensional representation.

The syntax of μZ is given as follows. Let $x \in \mathbb{V}$ be variable symbols, and let $\rho \in \mathbb{T}$ be (term) constructor symbols. We have the syntactic categories of *expressions* e, a

[2] Higher-order logic [7], for instance, has a smaller number of language constructs.

subset of them, *patterns* p, and of *constraints* ϕ:

$$\begin{array}{l} e \in \mathbb{E} ::= x \mid \rho(\overline{e}) \mid \{e\} \mid e \cup e' \mid \{p \setminus \overline{x} \mid \phi\} \mid \mathsf{hom}_\delta e \\ p \in \mathbb{P} \subseteq \mathbb{E} \\ \phi \in \mathbb{C} ::= e \subseteq e' \mid \mathsf{true} \mid \mathsf{false} \mid \phi \wedge \phi' \\ \delta \in \Delta == \{\mu, \mathsf{E}, \ldots\} \end{array}$$

The form $\rho(\overline{e})$ describes a term constructor application, where $\overline{e}$ is a sequence of expressions. (We generally use the convention to denote sequences of terms t by $\overline{t}$.) The set of patterns, $\mathbb{P} \subseteq \mathbb{E}$, are those expressions which are variables, x, or constructor applications to patterns, $\rho(\overline{p})$.

The form $\{e\}$ denotes a a singleton set, the form $e \cup e'$ set union. With $\{p \setminus \overline{x} \mid \phi\}$ a set comprehension is given: it denotes the set of values which match the pattern p such that there exists a solution for the local existential variables $\overline{x}$ such that the constraint ϕ is satisfied under the substitution of the match (constraints are described below). We will write $\{p \mid \phi\}$ for the case that $\#\overline{x} = 0$.

The last expression form, $\mathsf{hom}_\delta e$, denotes a *homomorphism* on the set e. In general, a homomorphism enumerates the elements of a set to compute a result from them. The μZ calculus provides a fixed (but extendible) number of builtin homomorphisms, whose treatment in the computation model is homogeneous. In this paper, we will explicitly use two of them:

- the μ-homomorphism determines the "μ-value" of a set. It is defined iff the set is not empty and contains no distinct elements; the unique element is then delivered.
- the E-homomorphism forces the extensional enumeration of elements of a set; it is defined iff the set is finitely enumeratable, then yielding the set itself.

$\mathsf{hom}_\delta e$ is the only expression form in our calculus which introduces undefinedness: the other expression forms can be interpreted as mere term constructions, whose interpretation is eventually demanded by an homomorphism.

We now take a closer look at the constraints used in set-comprehensions. We have tautologies, subset constraints, and constraint conjunction (denoted as $\phi \wedge \phi'$). The following abbreviations are used for constraints and expressions:

$$\begin{array}{ll} e \in e' == \{e\} \subseteq e' & 0 == \{x \mid \mathsf{false}\} \\ e = e' == e \in \{e'\} & 1 == \{x \mid \mathsf{true}\} \end{array}$$

The μZ calculus uses a shallow polymorphic type system in the ML style. From a set of basic type constructors for given and free types (which may be generic), higher types are build by powerset construction. Cartesian product is treated as a special (generic) free type constructor. Value constructors ρ are associated with an arity describing their argument types and their result type. Based on this, we can assume that expressions have a principal type. In the sequel, we only look at such well-formed expressions and ignore typing considerations.

As a tiny example, consider a constraint which defines the *append* function on lists. Lists are denoted by the constructors for the empty list, *nil*, and a list cell, $cons(x, xs)$.

Tuples are denoted by $(_, \ldots, _)$, a constructor application in mixfix notation:

$$\begin{aligned} append = {} & \{((nil, ys), ys) \mid \mathsf{true}\} \cup \\ & \{((cons(x, xs), ys), cons(x, zs)) \mid ((xs, ys), zs) \in append\} \end{aligned}$$

3 Mapping Z to μZ

Before we discuss the semantics and computation model of μZ, let us take a closer look on the mapping of Z to μZ. We do not give a formal and complete definition of the translation here, but only the most important equivalences.

3.1 Abstraction and Application

Z's set comprehension, λ-abstraction, and relational application is mapped as follows[3]:

$$\begin{array}{lcl} \{x : e \bullet e'\} & \leadsto & \{y \setminus x \mid x \in e \wedge y = e'\} \\ \lambda x : e \bullet e' & \leadsto & \{(x, y) \mid x \in e \wedge y = e'\} \\ e\, e' & \leadsto & \mathsf{hom}_\mu \{y \mid (e', y) \in e\} \end{array}$$

This handles functions as sets of pairs. The mapping of the application also captures relations: it is sufficient that e is a binary relation which is right-unique and defined just at the point e'.

As we will see later on, μZ computation model uses applicative order reduction for expressions, taking set comprehensions to be in normal form. But in the above mapping of $\lambda x : e \bullet e'$, for example, the domain e is shifted in the comprehension, and thus not strictly evaluated. In order to enforce a strict evaluation of the domain, we can use the following alternative mapping[4]:

$$\lambda x : e \bullet e' \quad \leadsto \quad \mathsf{hom}_\mu \{f \setminus t \mid t = e \wedge f = \{(x, y) \mid x \in t \wedge y = e'\}\}$$

3.2 Predicates

We look at the mapping of Z predicates in a context of a μZ set comprehension, $\{p \setminus \overline{x} \mid \mathcal{C}\, q\}$, where $\mathcal{C}$ is a conjunctive context (a sequence of one or more conjuncted constraints), and q, q' are not yet mapped Z predicates (the restriction to a comprehension context is not a real one, since an entire Z specification is treated as a top-level set comprehension). In order to represent truth values, sets over a singleton type are used, whose element is constructed by tt, such that truth is $\{\mathsf{tt}\}$ and falsity $\mathsf{0}$:

$$\begin{array}{lcl} \{p \setminus \overline{x} \mid \mathcal{C}(q \vee q')\} & \leadsto & \{p \setminus \overline{x} \mid \mathcal{C}(\mathsf{tt} \in \{\mathsf{tt} \mid q\} \cup \{\mathsf{tt} \mid q'\})\} \\ \{p \setminus \overline{x} \mid \mathcal{C}(q \Rightarrow q')\} & \leadsto & \{p \setminus \overline{x} \mid \mathcal{C}(\{\mathsf{tt} \mid q\} \subseteq \{\mathsf{tt} \mid q'\})\} \\ \{p \setminus \overline{x} \mid \mathcal{C}(\neg\, q)\} & \leadsto & \{p \setminus \overline{x} \mid \mathcal{C}(\{\mathsf{tt} \mid q\} \subseteq \mathsf{0})\} \\ \{p \setminus \overline{x} \mid \mathcal{C}(\exists y : e \mid q \bullet q')\} & \leadsto & \{p \setminus \overline{x}, y \mid \mathcal{C}(y \in e \wedge q \wedge q')\} \\ \{p \setminus \overline{x} \mid \mathcal{C}(\forall y : e \mid q \bullet q')\} & \leadsto & \{p \setminus \overline{x} \mid \mathcal{C}(\{y \mid y \in e \wedge q\} \subseteq \{y \mid q'\})\} \end{array}$$

[3] For ease of readability, we ignore name clash problems in the presentation – which can be tackled in the usual ways.

[4] Our compiler uses this mapping for λ-abstraction. In general, it compiles expressions appearing in the declaration part of Z schema text to become strictly evaluated.

Here, the introduction of the general subset-operator (instead of its special version, $e \in e'$) indicates the critical predicates regarding computability (negation, implication, and universal quantification). A general subset-relation requires the most sophisticated resolution techniques, and is not always computable in our model, as we will see later on.

3.3 Schema Calculus

In Z, a schema denotes a set of *bindings*, where bindings are tuples (records) with named components. If we treat bindings simply as term constructors, the schema calculus maps easily to μZ. Let the binding constructor of, for example, the Z schema $[a, b : \mathbb{N} \mid a < b]$ be $\langle\!| a == _, b == _ |\!\rangle$. The given schema can be represented as a μZ set comprehension

$$\{\langle\!| a == x, b == y |\!\rangle \mid x \in \mathbb{N};\ y \in \mathbb{N};\ (x, y) \in (_ < _)\}$$

where we used mixfix notation for the binding constructor.

In order to explain the operators of the schema calculus, we use $\Sigma(e)$ to denote a pattern representing a binding of e. With $\Sigma(e) \curlywedge \Sigma(e')$ we construct the pattern for the "joined" signatures of e and e'. $\Sigma(e)'$ denotes the "primed" version of a binding pattern for e:

$$\begin{array}{lcl} e \wedge e' & \rightsquigarrow & \{\Sigma(e) \curlywedge \Sigma(e') \mid \Sigma(e) \in e \wedge \Sigma(e') \in e'\} \\ e \vee e' & \rightsquigarrow & \{\Sigma(e) \curlywedge \Sigma(e') \mid \Sigma(e) \in e\} \cup \{\Sigma(e) \curlywedge \Sigma(e') \mid \Sigma(e') \in e'\} \\ e' & \rightsquigarrow & \{\Sigma(e)' \mid \Sigma(e) \in e\} \end{array}$$

In a similar way, all the other operators of the schema calculus can be mapped.

3.4 Types and Genericity

A Z given type, $[T]$, generates an according 0-ary type constructor in μZ's type system. In the context where the given type is introduced, the constraint $T = 1$ is added.

A Z free type, $T ::= c\langle\!\langle e \rangle\!\rangle \mid \ldots$ introduces the given type T in μZ's type system and constructors ρ_c with the according arity. It generates the constraints $T = 1$ and $c = \{(x, y) \mid x \in e \wedge y = \rho_c(x)\}$.

Z's genericity is more general then the shallow polymorphism of μZ, since constrained instances can be denoted. Consider e.g. the generic definition $id[X] == \{x : X \bullet (x, x)\}$ of the identity relation. Then $id[\{1, 2\}]$ is a possible instantiation and denotes the set $\{(1, 1), (2, 2)\}$. Generic definitions of Z need therefore to be mapped to *functions* over the generic's instance. The identity function is thus represented in μZ as $id = \{(X, id') \mid \mathsf{id}' = \{(x, x) \mid x \in X\}\}$. The application of this generic name in Z, $id[e]$, is function application in μZ, $\mathsf{id}\ e$, thus eventually the literal form $\mathsf{hom}_\mu\{y \mid (e, y) \in id\}$, which delivers the identity relation instantiated with e.

In many applications of genericity in Z, the instantiation of generics is automatically derived by the type checker. These instances are unconstrained – they are given types, power-sets of unconstrained types, and so on. It is subject of an easy to perform partial evaluation on μZ expressions to exploit such cases: for example, $id\,T$, where T is given type, is equivalent to $id\,1$, which after unsugaring and unfolding yields in $\mathsf{hom}_\mu\{y \mid$

$(1, y) \in \{(X, id') \mid \mathsf{id}' = \{(x, x) \mid x \in X\}\}$, which results in $\mathsf{hom}_\mu\{y \mid y = \{(x, x) \mid x \in 1\}\}$, and finally (by an instance of the "one-point-rule" for eliminating the μ-homomorphism) in $\{(x, x) \mid \mathsf{true}\}$.

4 Semantics

The model of μZ is a hierarchical typed universe, where sets are represented as *partial characteristic (boolean) functions* in a set-theoretical meaning, which can be founded by Z itself. This representation of sets is a generalization of partial three-point logics (see e.g. [18]) to the case of set-algebra. There are three observation to be made about a set represented as a partial characteristic function: an element is member (it is in the domain and is mapped to true), an element is not member (it is in the domain and is mapped to false), or it is unknown to be member or not (it is not in the domain). For example, consider the expression $\{x \mid ((1, x), 1) \in \mathsf{div}\}$: for 0, membership is unknown, for all other numbers it is known (1 is member and the other numbers are not).

The representation of sets allows for a natural definition of the set-algebraic operations. Let f_1, f_2 be the encoding of two sets in Z as functions $A \pfun \mathbb{P}[]$ (where booleans are represented as the empty schema, with $\varnothing$ falsity and $\{\langle\rangle\}$ truthness). Set intersection and set union are defined in our model as follows:

$$f_1 \cap f_2 = \lambda x : (\mathsf{dom}\, f_1 \cap \mathsf{dom}\, f_2) \cup (f_1^\sim)(\!|\{\mathsf{false}\}|\!) \cup (f_2^\sim)(\!|\{\mathsf{false}\}|\!) \bullet [\mid f_1\, x \wedge f_2\, x]$$
$$f_1 \cup f_2 \equiv \lambda x : (\mathsf{dom}\, f_1 \cap \mathsf{dom}\, f_2) \cup (f_1^\sim)(\!|\{\mathsf{true}\}|\!) \cup (f_2^\sim)(\!|\{\mathsf{true}\}|\!) \bullet [\mid f_1\, x \vee f_2\, x]$$

Thus, as sets with unknowns are combined with other sets, unknowns may vanish. On intersection, falsity dominates undefinedness and on union, truthness dominates undefinedness.

Since we have union and intersection as total functions on sets, a μZ powerset domain constitutes a *complete lattice*, with the empty set (the total characteristic function constantly delivering false) the bottom element and the universal set of the according type the top element. The *undefined* set (that is the characteristic function with the empty domain) is the bottom of a sub-lattice which starts in the "middle" of this lattice. In the lattice of a powerset, we can expect to construct fixed-points. Note that this does not necessarily mean that we can *compute* every construct of μZ (this would require continuity or at least monotonicity of all constructs): the μZ calculus and its semantic model is not constructive *a priori* – otherwise we were not able to map full Z to the calculus. Not restricting the calculus to computability has the advantage that we can give μZ specifications a meaning which are not entirely computable, but where our computation goal is concerned only with a computable subpart.

A rough edge in μZ's model is the treatment of free types. Given a type constructor t and a value constructor $\rho : \ldots \times \mathbb{P}\, t \longrightarrow t$, we know that the constructor cannot be free in a set-based model. We need to restrict its domain to the finite subsets of T, or the finite approximation of some chain in a lattice. Thus constructors for recursive free types are actually *partial injective* functions regarding the domain of their arity in μZ's type system.

5 Computation Model

A computation model for μZ is described in the style of natural semantics [16]. The intention is to get a model which is abstract enough to provide an idea of computation, but also concrete enough to serve as a starting point for an efficient implementation.

The computation model is not primarily designed with the goal to get "maximal" computation power out of μZ (a complete model cannot be realized anyway). Efficient implementability and transparent traceability of execution are as significant as computation power. The last point is in particular important for the application to test-case evaluation, where the reason of the failure of a test needs to be understandable by humans. Two major design decision are derived from these considerations:

- we use a *strict* (applicative) evaluation order. Strictness in the context of μZ means that members of the singleton set, $\{e\}$, are completely evaluated before the set is constructed. Similarly, in constructed values, $\rho(\overline{e})$, the arguments are fully evaluated prior to the constructor call.
- we will use *depth-first*, left-to-right resolution. Though our model can be easily extended to breadth-first, an implementation with breadth-first search is considerable more complex and less efficient.

Though depth-first search is used, the resolution techniques go behind those of Prolog, since the computation model provides full "parallel" execution of conjunctive constraints. Thus, when the constraint of $\{p \mid \phi_1 \wedge \phi_2\}$ is executed, and one of the ϕ_i's diverges, the result is nevertheless determined if the other constraint fails.

5.1 Domains

Values of the computation model are a normal form of expressions as specified by the following grammar:

$$v \in \mathbb{E}_V ::= \rho(\overline{v}) \mid \{v\} \mid \{p \setminus \overline{x} \mid \phi\} \mid v \cup v' \mid x$$

Thus in values all applications of the $\mathsf{hom}_\delta e$ form not appearing inside of a set-comprehension are eliminated.

Values can be "cyclic", that is regularly infinite. It is well known how cyclic terms can be represented by acyclic structures (for example, by a pair of a variable and value assigned to this variable, where the variable may appear in the value) and we thus use them without explicit formalization.

Variables contained in a value can be "frozen". Freezing a variable x maps it to a unique variable x_f. With $\mathsf{freeze}\ X\ v$ the variables $x \in X$ are frozen in v, with $\mathsf{unfreeze}\ X\ v$ they are unfrozen. The function $\mathsf{free}\ v$ delivers those free variables in a value which are not frozen, the function $\mathsf{frozen}\ v$ those which are. The purpose of freezing variables is to treat them as *constants* regarding unification in certain contexts, as will be discussed below.

A *substitution* is a mapping from variables to values, defined in the usual way. With $\mathsf{bound}\,\sigma = \{x : \mathsf{dom}\,\sigma \mid \sigma x \neq x\}$ the bound variables of a substitution are denoted.

It holds $\mathsf{bound}\,\sigma \cap \bigcup((\mathsf{free} \cup \mathsf{frozen})(\!|\mathsf{ran}\,\sigma|\!)) = \emptyset$. With $\sigma[x := v]$ we extend a substitution by a variable assignment – the value v may contain references to x, which are converted to cyclic terms. With $\sigma \circ \sigma'$ we denote composition of substitutions.

A (partial) *equality* on values, written as $v \sim v'$, is available. Variables (frozen or not) are equal only by name. Set comprehensions are equal by syntactic structure (where no renaming of variables is involved)[5]. The equality takes commutativity, associativity and idempotency of set union into account, and handles cyclic terms. Furthermore, we have the relation $v \not\sim v'$ to indicate inequality of values. Note that this is *not* the reverse of $v \sim v'$, since inequality of variables of different names cannot be decided, as well as inequality of set comprehensions.

A few further auxiliary definitions are required. In the computation rules, we use the notion of a *goal*, a substitution paired with a constraint, written as $\theta = \sigma :: \phi$. A *choice* is a sequence of goals, $\overline{\theta} = \langle \theta_1, \ldots, \theta_n \rangle$. The propagation of a substitution over a choice is defined as

$$\sigma \rhd \langle \sigma_1 :: \phi_1, \ldots, \sigma_n :: \phi_n \rangle = \langle \sigma \circ \sigma_1 :: \phi_1, \ldots, \sigma \circ \sigma_n :: \phi_n \rangle$$

5.2 Reduction

Computation is defined by two relations which are mutually dependent. The first relation, $e \xrightarrow{\sigma} e'$, describes a reduction step on expressions under the substitution σ. The second, $\overline{\theta} \rightsquigarrow \overline{\theta}'$, describes a resolution step by mapping a choice into a choice.

The rules for $e \xrightarrow{\sigma} e'$ are given in Fig. 1. Here, $\mathcal{S}$ describes a strict reduction redex, which is specified by a "grammar with a hole". Every expression which is not embedded in a set-comprehension is strict. Rule $E1$ defines reduction in such contexts. Rule $E2$ describes the unfolding of a bound variable.

The remaining rules for expression reduction handle homomorphisms. To abstract from the particular kind of homomorphism, we use the following characteristic functions for homomorphisms:

- $hs = \mathit{start}\,\delta$ describes the initial value of an internal state hs which is used to represent accumulated information as the elements of a set are stepwise enumerated.
- $hs' = \mathit{next}\,\delta\,(hs, v)$ describes how the state hs is transformed for the next element v. In general, $\mathit{next}\,\delta$ might be a partial function, where $(hs, v) \notin \mathsf{dom}(\mathit{next}\,\delta)$ means that for the according state hs and value v no transition is possible.
- $v' = \mathit{stop}\,\delta\,(hs, v)$ describes that the reduction of δ can be stopped, since enough information for computing the result v' is available. For example, a homomorphism representing a test whether a set is non-empty can immediately stop when the first solution is encountered. Again, $\mathit{stop}\,\delta$ is partial, the domain test indicating when it can be applied.
- $v = \mathit{end}\,\delta\,hs$ describes how the final result is computed from the state hs after all elements have been enumerated. *end* may be partial.

In rules $E3$ to $E7$ in Fig. 1, we use the intermediate expression form $\mathsf{HOM}_\delta(hs, X, x, \overline{\theta})$ to represent the state of homomorphism reduction: hs is the current internal state, X a

[5] In the implementation, this kind of equality can be easily detected by pointer equality.

Fig. 1 Expression Reduction

$$\mathcal{S}\cdot ::= (\mathcal{S}\cdot)\cup e \mid e\cup(\mathcal{S}\cdot) \mid \{\mathcal{S}\cdot\} \mid \rho(\ldots,\mathcal{S}\cdot,\ldots) \mid \mathsf{hom}_\delta(\mathcal{S}\cdot) \mid \cdot$$

$$E1\,\frac{e \xrightarrow{\sigma} e'}{\mathcal{S}\ e \xrightarrow{\sigma} \mathcal{S}\ e'} \qquad E2\,\frac{x \in \mathsf{bound}\,\sigma}{x \xrightarrow{\sigma} \sigma x}$$

$$E3\,\frac{X = \mathsf{free}\,v;\ x \notin X \cup \mathsf{frozen}\,v}{\mathsf{hom}_\delta\,v \xrightarrow{\sigma} \mathsf{HOM}_\delta(\mathit{start}\,\delta, X, x, \langle\sigma :: \{x\} \subseteq \mathsf{freeze}\ X\,v\rangle)}$$

$$E4\,\frac{\sigma \rhd \overline{\theta} \rightsquigarrow \overline{\theta}'}{\mathsf{HOM}_\delta(hs, X, x, \overline{\theta}) \xrightarrow{\sigma} \mathsf{HOM}_\delta(hs, X, x, \overline{\theta}')}$$

$$E5\,\frac{(hs, \sigma'\,x) \in \mathsf{dom}(\mathit{next}\,\delta)}{\mathsf{HOM}_\delta(hs, X, x, \langle\sigma' :: \mathsf{true}\rangle \frown \overline{\theta}) \xrightarrow{\sigma} \mathsf{HOM}_\delta(\mathit{next}\,\delta\ (hs, \sigma'\,x), X, x, \overline{\theta})}$$

$$E6\,\frac{(hs, \sigma'\,x) \in \mathsf{dom}(\mathit{stop}\,\delta)}{\mathsf{HOM}_\delta(hs, X, x, \langle\sigma' :: \mathsf{true}\rangle \frown \overline{\theta}) \xrightarrow{\sigma} \mathsf{unfreeze}\ X\ (\mathit{stop}\,\delta\ (hs, \sigma'\,x))}$$

$$E7\,\frac{hs \in \mathsf{dom}(\mathit{end}\,\delta)}{\mathsf{HOM}_\delta(hs, X, x, \langle\rangle) \xrightarrow{\sigma} \mathsf{unfreeze}\ X\ (\mathit{end}\,\delta\ hs)}$$

set of frozen context variables and $\overline{\theta}$ a choice which represents possible solutions for the variable x. In Rule $E3$, we initialize this intermediate form: hs is set to $\mathit{start}\,\delta$, and the choice to $\langle\sigma :: x \in \mathsf{freeze}\ X\,v\rangle$, where x is a fresh variable and X the set of free variables contained in the set-value v. These variables need to be frozen (which will be discussed below).

The choice embedded in $\mathsf{HOM}_\delta(hs, X, x, \overline{\theta})$ is reduced by rule $E4$. The substitution of the expression reduction context is propagated over $\overline{\theta}$, in order to make the current bindings of frozen context variables available.

Each time a solution for $\overline{\theta}$ is found, we can either continue by applying $\mathit{next}\,\delta$ (Rule $E5$), or stop enumerating solutions with $\mathit{stop}\,\delta$ (Rule $E6$). $\mathit{end}\,\delta$ is invoked if all solutions are enumerated (Rule $E7$). The context variables in the result of $\mathit{end}\,\delta$ and $\mathit{stop}\,\delta$ are unfrozen.

Note that it might be possible that none of the rules $E5$ to $E7$ is applicable, since the functions characterizing the homomorphism are not defined for the current reduction state. In this case, our expression is irreducible, which we identify with "undefined". A further source of undefinedness is divergence, which happens if sets are enumerated infinitively.

Characterizing Basic Homomorphisms. Here are the definitions of the characteristic functions for the E morphism, which computes an extensional representation of a set (if it exist), and of the μ homomorphism. In the definitions we assume that for all unmentioned cases arguments are not in the domain of the according functions. We give no cases for the *stop* function (implying that it is the totally undefined function) since both

Fig. 2 Constraint Resolution Rules: Selection and Reduction

$$\mathcal{C}\cdot ::= (\mathcal{C}\cdot)\wedge\phi \mid \phi\wedge(\mathcal{C}\cdot) \mid \cdot \qquad C1\frac{\langle\sigma :: e\subseteq e'\rangle \rightsquigarrow \langle\sigma_1 :: \phi_1,\ldots,\sigma_n :: \phi_n\rangle}{\langle\sigma :: \mathcal{C}(e\subseteq e')\rangle \frown \overline{\theta} \rightsquigarrow \langle\sigma_1 :: \mathcal{C}\,\phi_1,\ldots,\sigma_n :: \mathcal{C}\,\phi_n\rangle \frown \overline{\theta}}$$

$$C2\frac{i,j\in\{1,2\};\ i\neq j;\ \phi_i = \mathsf{true}}{\langle\sigma :: \mathcal{C}(\phi_1\wedge\phi_2)\rangle \frown \overline{\theta} \rightsquigarrow \langle\sigma :: \mathcal{C}\,\phi_j\rangle \frown \overline{\theta}} \qquad C3\frac{e_i \xrightarrow{\sigma} e_i';\ i,j\in\{1,2\};\ i\neq j;\ e_j' = e_j}{\langle\sigma :: e_1\subseteq e_2\rangle \rightsquigarrow \langle\sigma :: e_1'\subseteq e_2'\rangle}$$

homomorphisms need to enumerate the entire set:

$$\begin{array}{lll} \mathit{start}\,\mathsf{E} & = 0 & \\ \mathit{next}\,\mathsf{E}\,(v,v') & = v\cup\{v'\} & \\ \mathit{end}\,\mathsf{E}\,v & = v & \end{array} \qquad \begin{array}{lll} \mathit{start}\ \mu & = \varnothing & \\ \mathit{next}\ \mu\ (\varnothing,v) & = \{v\} & \\ \mathit{next}\ \mu\ (\{v\},v') & = \{v\}\ \textbf{if}\ v\sim v' & \\ \mathit{end}\ \mu\ \{v\} & = v & \end{array}$$

Note that homomorphism functions in general need to be right-commutative, since the order in which the elements of a set are enumerated is not determined.

Frozen Variables. During homomorphism reduction, variables from the context of the reduction need to be frozen, preventing that resolution (as described in the next section) can generate assignments for them. To understand the reason for freezing, consider an expression like $\{y \mid \mathsf{hom}_{\mathsf{E}}\{\mathsf{tt} \mid y = 1\} \subseteq 0\}$ – which is a possible literal representation of $\{y \mid \neg\ y = 1\}$. Clearly, during the reduction of the homomorphism, we are not allowed to generate bindings for y. Instead, constraints in negated contexts must residuate until required context variables are bound.

5.3 Resolution

Having defined expression *reduction*, we now look at the rules for constraint *resolution*, which spawn the relation $\overline{\theta} \rightsquigarrow \overline{\theta}'$, mapping a choice into a choice.

Selection and Reduction. The first group of rules describes the selection of the subset constraint to be next reduced, and the reduction of expressions in subset constraints (Fig. 2). Let $\mathcal{C}$ describe a conjunctive context. Rule $C1$ uses $\mathcal{C}\ \phi$ to select some constraint to be reduced next from the head goal of the choice. Note that by the definition of $\mathcal{C}$ this selection is non-deterministic – indicating that constraints in conjunctions are *concurrently* reduced. Note moreover, since we only select from the head goal, our resolution strategy resembles depth-first search.

The selected subset constraint in Rule $C1$ is rewritten in the rule's premise into a choice which is not necessarily singleton. Propagating the context over this several possibilities in the result can be actually realized by backtracking in an implementation.

Rule $C2$ in Fig. 2 eliminates tautologies, Rule $C3$ handles expression reduction in subset constraints. Before actual resolution can start, expressions are reduced to a normal form, such that all homomorphisms are eliminated, and bound variables in strict contexts are substituted.

Fig. 3 Constraint Resolution Rules: Unification and Equality

$$C4\ \frac{}{\langle \sigma :: \{\rho(v_1,\ldots,v_n)\} \subseteq \{\rho(v'_1,\ldots,v'_n)\}\rangle \rightsquigarrow \langle \sigma :: \{v_1\} \subseteq \{v'_1\} \wedge \ldots \wedge \{v_n\} \subseteq \{v'_n\}\rangle}$$

$$C5\ \frac{\rho \neq \rho'}{\langle \sigma :: \{\rho(\ldots)\} \subseteq \{\rho'(\ldots)\}\rangle \rightsquigarrow \langle\rangle} \qquad C6\ \frac{\begin{array}{c} i,j \in \{1,2\};\ i \neq j \\ v_i \textit{ not frozen variable};\ v_i \notin \mathsf{bound}\,\sigma \end{array}}{\langle \sigma :: \{v_1\} \subseteq \{v_2\}\rangle \rightsquigarrow \langle \sigma[v_i := v_j] :: \mathsf{true}\rangle}$$

$$C7\ \frac{v \sim v'}{\langle \sigma :: \{v\} \subseteq \{v'\}\rangle \rightsquigarrow \langle \sigma :: \mathsf{true}\rangle} \qquad C8\ \frac{v \not\sim v'}{\langle \sigma :: \{v\} \subseteq \{v'\}\rangle \rightsquigarrow \langle\rangle}$$

$$C9\ \frac{i,j \in \{1,2\};\ i \neq j;\ v_i \textit{ non-extensional set};\ v'_i = \mathsf{hom}_{\mathsf{E}} v_i;\ v'_j = v_j}{\langle \sigma :: \{v_1\} \subseteq \{v_2\}\rangle \rightsquigarrow \langle \sigma :: \{v'_1\} \subseteq \{v'_2\}\rangle}$$

Fig. 4 Constraint Resolution Rules: Membership and Inclusion

$$C10\ \frac{}{\langle \sigma :: 0 \subseteq v\rangle \rightsquigarrow \langle \sigma :: \mathsf{true}\rangle} \qquad C11\ \frac{}{\langle \sigma :: \{v\} \subseteq 0\rangle \rightsquigarrow \langle\rangle}$$

$$C12\ \frac{\textit{rename } p,\overline{x},\phi \textit{ wrt } \sigma}{\langle \sigma :: \{v\} \subseteq \{p \setminus \overline{x} \mid \phi\}\rangle \rightsquigarrow \langle \sigma :: \{v\} \subseteq \{p\} \wedge \phi\rangle}$$

$$C13\ \frac{}{\langle \sigma :: v_1 \cup v_2 \subseteq v\rangle \rightsquigarrow \langle \sigma :: v_1 \subseteq v \wedge v_2 \subseteq v\rangle}$$

$$C14\ \frac{}{\langle \sigma :: \{v\} \subseteq v_1 \cup v_2\ \rangle \rightsquigarrow \langle \sigma :: \{v\} \subseteq v_1, \sigma :: \{v\} \subseteq v_2\rangle}$$

$$C15\ \frac{}{\sigma :: \{p \setminus \overline{x} \mid \phi\} \subseteq v \rightsquigarrow \sigma :: \mathsf{hom}_{\mathsf{E}}\{p \setminus \overline{x} \mid \phi\} \subseteq v}$$

Unification and Equality. The rules given in Fig. 3 describe the resolution of equalities, that is subset constraints of the form $\{v\} \subset \{v'\}$, by unification. Rule $C4$ models the decomposition of constructor terms, Rule $C5$ the case where constructors are detected to be unequal. Rule $C6$ handles variable assignment: only non-frozen, free variables can be assigned. Note that no occurs-check is performed.

The remaining rules are for the case that we look at equality between *sets* (that is something like $\{\{\ldots\}\} \subseteq \{\{\ldots\}\}$). If we can decide this equality by use of the approximative relations $_\sim_$ (respectively $_\not\sim_$), we have done. However, if the sets are not extensional, we need to convert them to an extensional representation, which is driven by Rule $C9$, inserting a $\mathsf{hom}_{\mathsf{E}} v_i$ expression.

Membership and Inclusion. The rules in Fig. 4 define the resolution of membership and general inclusion constraints. Rules $C10$ and $C11$ handle tautologies. In Rule $C12$, $\{v\} \subseteq v'$ ($v \in v'$) is treated for the case that $v' = \{p \setminus \overline{x} \mid \phi\}$. The set comprehension v' is instantiated in the resolution context by adding the equality $\{v\} \subseteq \{p\}$ and the constraint ϕ, after renaming its variables away from the variables of the context.

Rule $C13$ reduces a set union on the left-hand side of a subset constraint into a conjunction of subset constraints. Rule $C14$ reduces a union at the right-hand side, thereby creating a choice. $C14$ is the only source of choice creation, and corresponds to rule selection in logic languages. For example, consider the *append* relation given in Sec. 2, defined as a union of two set comprehensions, $term \cup recur$, one describing the termination case, the other the recursive case. For resolving $(xs, ys, zs) \in term \cup recur$, we create the choices $(xs, ys, zs) \in term$ and $(xs, ys, zs) \in recur$.

Rule $C15$, finally, handles a set-comprehension on the left-hand side of a subset constraint: this can be resolved only by extensional enumeration such that afterwards $C13$ can take over.

Confluence, Totalness and Strategies. An overlapping between the applicability of rules is present between the equality checks in $C7/C8$ and some other rules. However, in these cases the equality rules are only shortcuts which yield in similar results. In principle, a further candidate source for non-confluence is the non-deterministic selection of the next subset constraint from the head of a choice (Rule $C1$) – but in a conjunction of subset constraints it is obvious that this non-determinism introduces no problems for confluence.

As seen from the syntactic shapes of the resolution rules, we cover all cases of subset constraints where the left and the right-hand side are not variables. Thus our resolution is not total. Since expressions can be irreducible, resolution is not total from this end as well. Moreover, for resolving some subset constraints it is necessary to introduce new expressions, converting intensional descriptions by an homomorphism into extensional ones (Rules $C9$ and $C15$). The failure of extensionalization of sets is the source of failure of resolution of these subset constraints.

Though our system of resolution rules is confluent, it makes a big difference regarding *efficiency* in which order the rules are applied. In an implementation, we employ the so-called Andorra Principle [22], suspending the creation of choices (Rule $C14$) as long as possible, and preferring deterministic for indeterministic computation, a technique which is known to prune the search space dramatically for many applications.

5.4 Implementation

The computation model of μZ implies the design of an abstract machine [9], called ZAM, which has been implemented in C++ as part of the ZETA system. The ZAM is a *concurrent* constraint resolution machine, which has some similarities with the machine described for Oz in [17], but inherits its particular set-based flavor from the μZ calculus. Since it is a byte-code machine, concurrent constraint resolution can be implemented quite efficiently using software threads. Threads basically use logic variables for communication. Trailing and backtracking can be efficiently realized. The Andorra Principle is implemented by using thread priorities. More details are found in [8, 9].

6 What Can Be Computed?

Our model has limitations, some of them by design, others because the right (efficient) resolution techniques are not yet integrated. But what Z specifications, as they

are mapped to μZ, can be actually computed and what not? In this section, we give some examples and also identify some restrictions and possible enhancements for the future.

6.1 The Positive Answer

To central answer to the positive version of the question is by analogy: we can compute all constructs of Z which correspond to functional languages, as well as all constructs which correspond to logic languages such as Prolog. We look at some examples to illustrate these capabilities.

Logic Style. Set objects – relations or functions – are executable if they are defined by (recursive) equations, as in the following example, where we define natural numbers as a free type, addition on these numbers and an order relation:

$$
\begin{array}{l}
N ::= Z \mid S\langle\langle N \rangle\rangle \qquad\qquad \mid three == S(S(S(Z))) \\
\mid add : \mathbb{P}((N \times N) \times N) \\
\mid add = \{y : N \bullet (Z, y) \mapsto y\} \cup \\
\qquad \{x, y, z : N \mid (x, y) \mapsto z \in add \bullet (S\,x, y) \mapsto S\,z\} \\
\mid less == \{x, y : N \mid \exists t : N \bullet (x, S\,t) \mapsto y \in add\}
\end{array}
$$

In the ZETA system, we may now execute queries such as the following, where we ask for the pair of sets less and greater than *three*:

$$(\{x : N \mid (x, three) \in less\}, \{x : N \mid (three, x) \in less\})$$

```
⇛ ({Z,S(Z),S(S(Z))},{S(S(S(S(t))))})
```

Note that the second value of the resulting pair is a singleton set containing the free variable *t*. These capabilities are similar to logic programming. In fact, we can give a translation from any clause-based system to a system of recursive set-equations in the style given for *add*, where we collect all clauses for the same relational symbol into a union of set-comprehensions, and map literals $R(e_1, \ldots, e_n)$ to membership tests $(e_1, \ldots, e_n) \in R$.

Functional Style. The elegance of the functional paradigm comes from the fact that functions are first-class citizens. In our implementation of execution for Z, sets are first-class citizens as well. For example, we can execute operators such as relational image on base of their natural definition:

$$
\begin{array}{|l}
\hline
[X, Y] \\
_(\!|_|\!) == \lambda R : \mathbb{P}(X \times Y);\ S : \mathbb{P}\,X \bullet \{x : X;\ y : Y \mid x \in S \land (x, y) \in R \bullet y\} \\
\hline
\end{array}
$$

We can now query for the relational image of the *add* function over the cartesian product of the numbers less then three:

$$\mathbf{let}\ ns == \{x : N \mid (x, three) \in less\} \bullet add(\!|ns \times ns|\!)$$

$\Rightarrow$ `{Z,S(Z),S(S(Z)),S(S(S(Z))),S(S(S(S(Z))))}`

It is also possible to define the arrow types of Z, as shown below for the set of partial functions:

$$[X, Y]$$
$$_ \pfun _ == \{R : \mathbb{P}(X \times Y) \mid (\forall x : X \mid x \in \mathsf{dom}\, R \bullet \exists_1 y : Y \bullet (x, y) \in R)\}$$

This example makes use of universal and unique existential quantification, which are a possible source of non-executability in our setting. As seen in Sec. 3, the predicate $\forall x \mid p \bullet q$ is mapped to the subset constraint $\{x \mid p\} \subseteq \{x \mid q\}$. A subset constraint with a set-comprehension on the left-hand side is only resolvable if the set-comprehension can be enumerated by the E-homomorphism. Thus, if we try to check whether *add* is a function, we get in a few seconds:

$add \in N \times N \pfun N$

$\Rightarrow$
```
still searching after 200000 steps
gc #1 reclaimed 28674k of 32770k
...
```

In enumerating the domain of *add* our computation diverges. However, if we restrict *add* to a finite domain it works:

$\exists_1 ns == \{x : N \mid (x, three) \in less\} \bullet (ns \times ns) \lhd add \in N \times N \pfun N$

$\Rightarrow$ `*true*`

The examples show that Z's way to deal with functions and relations is fully supported: most toolkit functions are executable on base of their natural definition in Z. Obviously, by our mapping of the schema calculus to sets, also the usual sequential specification style of Z is fully supported.

Specifications. A Z specification is represented in μZ as a "top-level" set-comprehension containing as constraints all the axiomatic definitions of the incorporated Z sections[6]. We can execute such specifications even if they are loosely specified. Consider the following axiomatic definitions:

$$x, y : N$$
$$\hline$$
$$y \in \{z : N \mid (z, three) \in less\}$$

Querying for the pair (x, y) yields in:

(x, y)

[6] In order to support efficient separate compilation of Z sections, the reality is a little more complicated, but the conceptual view is as this.

```
⇛ (x,Z)
  (possibly more solutions)
  ...
  (x,S(Z))
  (possibly more solutions)
```

Thus the ZETA frontend allows us to step through the different "models" of a loose specification. The case that a constant is not constraint at all – in the example the x – causes no problems as long as its value is not required for other computations.

6.2 The Negative Answer (And What Can be Enhanced)

Declarations. The most obvious restriction when trying to execute a typical Z specification with our approach is that restrictions for set objects as often introduced by declarations are only executable if the objects are finitely enumeratable. The example of the partial function arrow and the divergence of the test $add \in N \pfun N$ in the previous section has illustrated this.

There are two pragmatic ways to deal with this. One is to treat all constraints introduced by Z declarations, $x : e$, as *assumptions*, which are discarded on their translation to μZ. A second is to demand that the user must explicitly markup assumed declarations, by using a special primitive function $\mathbf{assumed}_$, applied as in $add : \mathbf{assumed}\, N \times N \longrightarrow N$. This function is semantically identity, but treated specially by the compiler. Both approaches are currently supported by the ZETA system, controlled by user switches.

It has turned out in several applications that discarding declaration constraints seems to be the more practical approach. If a declared membership is actually a constraint required for execution, the user has to place it in the constraint part of schema text. However, this treatment violates the Z semantics which defines $add : N \times N \longrightarrow N$ to be equivalent to $add : \mathbb{P}((N \times N) \times N) \mid add \in N \times N \longrightarrow N$.

Definition Forms. When providing recursive definitions of relations and functions one currently has to stick to certain conventions. For example, the factorial function would be typically specified in Z by axioms of the kind:

$$fac\, 0 = 1;\ \forall x : \mathbb{N} \mid x > 0 \bullet fac(x) = x * fac(x-1)$$

A definition of this form is unamenable to execution. Instead one has to write:

$$fac = \{(0,1)\} \cup (\lambda x : \mathbb{N} \mid x > 0 \bullet x * fac(x-1))$$

The automatic conversion of axioms of the above form to definitions in the below form is subject of ongoing research, using techniques similar to those described in [3].

Powerset And The Like. The resolution techniques for general subset constraints in our model are relatively weak (they capture just functional and logic computation, not more). For example, we are not able to enumerate the powerset $\mathbb{P}\, A$, by its representation

in μZ as $\{x \mid x \subseteq A\}$, even if A is finitely enumeratable (indeed, we can test whether a ground value is in this set).

Technically, the constraint resolution system presented in this paper misses rules for the following kinds of subset constraints:

- subset constraints where the left or the right hand side is a free variable: $x \subseteq v$ or $v \subseteq x$. These constraints can generally not be resolved before x becomes bound. A typical instance is the powerset schema.
- constraints where the left-hand side is a set comprehension which fails to be enumeratable: $\{x \mid \phi\} \subseteq v$. These constraints are resolved by enumerating $\{x \mid \phi\}$ using $\mathsf{hom}_{\mathsf{E}}$. A typical instance is the universal quantor.
- subset constraints which represent unification problems on sets, for instance $\{\{x \cup y\}\} \subseteq \{\{1\} \cup \{2\}\}$. These constraints cannot produce bindings for x and y, since set-unification is currently not employed in our model. Instead, they must residuate until other constraints provide bindings.

There exist techniques which can (partially) solve these problems. An adaption of Gentzen's cut rule to constraint solving has been analyzed in [24] on base of the μZ calculus. Subset constraints are also subject of research in the realm of *set-based program analysis* (e.g. [1]), *subset logic programming* [14] and *set unification* (e.g. [2]). Our set-based model provides a framework for smooth integration of these techniques, which, however, needs further consolidation.

Resolution In Specific Domains. Though it is possible in our model to represent e.g. natural numbers by constructors 0 and $S\langle\langle N \rangle\rangle$, as shown in our toy examples, enabling resolution techniques on them, this approach is hopelessly inefficient for real applications. Thus numbers are actually integrated by a native implementation and resolution techniques for arithmetic constraints are not available. However, sophisticated solvers for e.g. linear arithmetics exist, as for many other specialized domains. A goal of future work is to provide an interface for integrating foreign solvers into our computation model and its implementation.

7 Conclusion and Related Work

Executing Z. Animation of the "imperative" part of Z is provided by the ZANS tool [15], imperative meaning Z's specification style for sequential systems using state transition schemas. This approach is highly restricted. An elaborated *functional approach* for executing Z has been described in [21], though no implementation exists today. A mapping of Z to Haskell is described in [6], where monads are used for dealing with the sequential specification style, but no logic resolution is employed. Mappings to Prolog are described, e.g., in [23]. Mappings of Z to Mercury are described in [25]: however, in this approach the effective power of execution is restricted compared with ours, because the data flow has to be determined statically. The approach presented in this paper goes beyond all the others, since it allows the combination of the functional and logic aspects of Z in a higher-order setting, treating sequential behaviors as first-class citizens, comparable as to the integration of imperative computation into functional languages using monads.

Implementation of Functional Logic Languages. The beauty of the set-based approach, which inherits μZ from Z, provides a way of viewing the λ-calculus, predicate calculus, schema's calculus, set-algebra, etc., in a unified, small expression language. Though the motivation of μZ and its implementation is the execution of Z, our approach is also interesting for the implementation of declarative programming languages in general, since μZ can embed functional logic languages such as Curry [12]. In its intention, the μZ calculus is comparable to the calculi given for Oz in [19].

The computation model we have given is comparable to the one described in [11]. Normalized set values correspond to the "definitional trees" used in this work. Higher-orderness has a cleaner representation in μZ, since set-values are first-class inhabitants of the operational domain which definitional trees are not. Moreover, our model supports encapsulated search by computation of the $\mathsf{hom}_\delta e$ form, which goes behind [11].

A strict reduction order has been chosen for a easier traceability of execution by humans, which is important for the application to test-data evaluation, and a supposed better overall performance. It is drawn from experiences with functional programming and language implementation that the overhead of laziness has to be paid for significantly, which in practice often results in that programmers use strictness annotations [13].

Acknowledgment. The discussions in our constraint group at the TU Berlin have contributed much to the presentation of μZ and its computation model in this paper. Thanks go to Petra Hofstedt, Markus Lepper and Jacob Wieland for the fruitful discussions. Moreover, the referees have provided useful information to enhance the paper.

References

1. A. Aiken and E. Wimmers. Solving systems of set constraints. In *Symposium on Logic in Computer Science*, pages 329–340, June 1992.
2. P. Arenas-Sanchez and A. Dovier. A minimality study for set unification. *Journal of Functional and Logic Programming*, 1997(7), 1997.
3. R. D. Arthan. Recursive definitions in Z. In J. P. Bowen, A. Fett, and M. G. Hinchey, editors, *ZUM'98: The Z Formal Specification Notation*, volume 1493 of *Lecture Notes in Computer Science*, pages 154–171. Springer-Verlag, 1998.
4. R. Büssow and W. Grieskamp. *The ZETA System Documentation.* Technische Universität Berlin, Dec. 1998. URL: `http://uebb.cs.tu-berlin.de/zeta`.
5. R. Büssow and W. Grieskamp. A Modular Framework for the Integration of Heterogenous Notations and Tools. In K. Araki, A. Galloway, and K. Taguchi, editors, *Proc. of the 1st Intl. Conference on Integrated Formal Methods – IFM'99*. Springer-Verlag, London, June 1999.
6. H. S. Goodman. The Z-into-Haskell tool-kit: An illustrative case study. In J. P. Bowen and M. G. Hinchey, editors, *ZUM'95: The Z Formal Specification Notation*, volume 967 of *Lecture Notes in Computer Science*, pages 374–388. Springer-Verlag, 1995.
7. M. Gordon and T. Melham, editors. *Introduction to HOL: A theorem proving environment for higher-order logics*. Cambridge University Press, 1993.
8. W. Grieskamp. The mZ calculus and its implementation. In *Proceedings of the International Workshop on Implementation of Declarative Languages (IDL'99)*, Sept. 1999.
9. W. Grieskamp. *A Set-Based Calculus and its Implementation.* PhD thesis, Technische Universität Berlin, 1999.
10. M. Hanus. The integration of functions into logic programming: From theory to practice. *Journal of Logic Programming*, 19(20), 1994.

11. M. Hanus. A unified computation model for declarative programming. In *Proc. of the 1997 Joint Conference on Declarative Programming*, Grado (Italy), 1997.
12. M. Hanus. Curry – an integrated functional logic language. Technical report, Internet, 1999. Language report version 0.5.
13. P. H. Hartel, M. Feeley, M. Alt, L. Augustsson, P. Baumann, M. Beemster, E. Chailloux, C. H. Flood, W. Grieskamp, J. H. G. van Groningen, K. Hammond, B. Hausman, M. Y. Ivory, R. E. Jones, J. Kamperman, P. Lee, X. Leroy, R. D. Lins, S. Loosemore, N. Röjemo, M. Serrano, J.-P. Talpin, J. Thackray, S. Thomas, P. Walters, P. Weis, and P. Wentworth. Benchmarking implementations of functional languages with "pseudoknot", a Float-Intensive benchmark. *J. of Functional Programming*, 6(4), 1996.
14. B. Jayaraman and K. Moon. Subset logic programs and their implementation. *Journal of Logic Programming*, 19(20), 1999.
15. X. Jia. An approach to animating Z specifications. Internet: `http://saturn.cs.depaul.edu/~fm/zans.html`, 1996.
16. G. Kahn. Natural semantics. In *Symposium on Theoretical Computer Science (STACS'97)*, volume 247 of *Lecture Notes in Computer Science*, 1987.
17. M. Mehl, R. Scheidhauer, and C. Schulte. An Abstract Machine for Oz. Research Report RR-95-08, Deutsches Forschungszentrum für Künstliche Intelligenz, Stuhlsatzenhausweg 3, D66123 Saarbrücken, Germany, June 1995. Also in: *Proceedings of PLILP'95*, Springer-Verlag, LNCS, Utrecht, The Netherlands.
18. O. Owe. Partial logic reconsidered: A conservative approach. *Formal Aspects of Computing*, 5:208–223, 1997.
19. G. Smolka. A calculus for higher-order concurrent constraint programming with deep guards. Research Report RR-94-03, Deutsches Forschungszentrum für Künstliche Intelligenz, Stuhlsatzenhausweg 3, D-66123 Saarbrücken, Germany, Feb. 1994.
20. J. M. Spivey. *The Z Notation: A Reference Manual*. Prentice Hall International Series in Computer Science, 2nd edition, 1992.
21. S. Valentine. The programming language Z^{--}. *Information and Software Technology*, 37(5–6):293–301, May–June 1995.
22. D. H. D. Warren. The extended andorra model with implicit control. In *ICLP'90 Parallel Logic Programming Workshop*, 1990.
23. M. M. West and B. M. Eaglestone. Software development: Two approaches to animation of Z specifications using Prolog. *IEE/BCS Software Engineering Journal*, 7(4):264–276, July 1992.
24. J. Wieland. A resolution algorithm for general subtyping constraints. Master's thesis, Technische Universität Berlin, 1999.
25. M. Winikoff, P. Dart, and E. Kazmierczak. Rapid prototyping using formal specifications. In *Proceedings of the Australasian Computer Science Conference*, 1998.